

MÉTODOS DE ARRAY EM JAVASCRIPT

`map()` • `filter()` • `find()`

🔪 1. Por que usar métodos de Array?

Um **Array** é uma lista de valores:

```
const numeros = [10, 20, 30, 40];
```

Os métodos de Array permitem **processar essa lista de forma simples**, evitando a necessidade de escrever vários `for` ou `while`.

Os três métodos principais estudados são:

- `map()` → **transformar**
- `filter()` → **filtrar**
- `find()` → **encontrar**

◆ 2. `map()` — TRANSFORMAR

O `map()` percorre **todos os elementos** e cria um **novo array com os elementos transformados**.

```
const numeros = [10, 20, 30];

const dobrados = numeros.map(numero => numero * 2);
```

Resultado:

```
[20, 40, 60]
```

☐ **Pense:**

MAP = transformar TODOS

A quantidade de elementos normalmente permanece a mesma:

```
[10, 20, 30]
      ↓ map()
[20, 40, 60]
```

3 → 3 elementos

◆ 3. `filter()` — FILTRAR

O `filter()` percorre o array e mantém **somente os elementos que atendem a uma condição**.

```
const idades = [15, 22, 17, 35, 19];

const maiores =
  idades.filter(idade => idade >= 18);
```

Resultado:

```
[22, 35, 19]
```

O método verifica cada elemento:

```
15 → ✖
22 → ✔
```

17 → ✖
35 → ✔
19 → ✔

☐ **Pense:**

FILTER = selecionar quem passa na regra

A quantidade de elementos pode diminuir.

◆ 4. `find()` — ENCONTRAR

O `find()` procura o **primeiro elemento** que atende a uma determinada condição.

```
const usuarios = [
  { nome: "Ana", admin: false },
  { nome: "João", admin: true },
  { nome: "Maria", admin: true }
];

const administrador =
  usuarios.find(usuario => usuario.admin);
```

Resultado:

```
{ nome: "João", admin: true }
```

O `find()` para quando encontra o **primeiro elemento correspondente**.

Se nenhum elemento for encontrado:

```
undefined
```

☐ **Pense:**

FIND = encontrar UM

📊 5. DIFERENÇA ENTRE OS TRÊS

Método	Pergunta que você deve fazer	Retorno
<code>map()</code>	"Quero transformar todos?"	Novo Array
<code>filter()</code>	"Quero selecionar alguns?"	Novo Array
<code>find()</code>	"Quero encontrar um?"	Elemento ou <code>undefined</code>

🔥 MACETE

MAP → TRANSFORMA

FILTER → FILTRA

FIND → ENCONTRA

☐ 6. Aplicação em QA

Imagine que temos resultados de testes:

```
const testes = [
  { nome: "Login", passou: true },
```

```
    { nome: "Logout", passou: false },
    { nome: "Cadastro", passou: false }
  ];
```

Encontrar todos os testes que falharam:

```
const falhas =
  testes.filter(teste => !teste.passou);
```

Resultado:

Logout
Cadastro

Encontrar a primeira falha:

```
const primeiraFalha =
  testes.find(teste => !teste.passou);
```

Resultado:

Logout

Isso é muito útil para analisar **resultados de testes automatizados**.

📦 7. COMO ESCOLHER O MÉTODO?

Pergunte:

Quero transformar todos?

➡ `map()`

```
numeros.map(n => n * 2);
```

Quero somente os que atendem a uma condição?

➡ `filter()`

```
numeros.filter(n => n > 20);
```

Quero encontrar apenas o primeiro que atende à condição?

➡ `find()`

```
numeros.find(n => n > 20);
```

🎓 RESUMO PARA A PROVA

`map()`

Transforma cada elemento e retorna um novo Array.

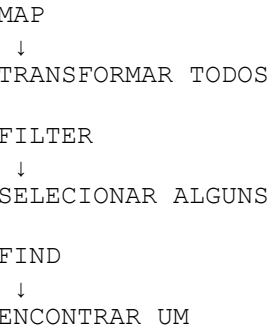
`filter()`

Seleciona os elementos que atendem a uma condição e retorna um novo Array.

`find()`

Retorna o primeiro elemento que atende a uma condição.

★ PARA MEMORIZAR



MAP transforma → FILTER seleciona → FIND encontra.

⚠ Atenção à diferença mais importante